

CS P51 - OPERATING SYSTEMS

UNIT – I

Introduction: Mainframe Systems – Desktop Systems – Multiprocessor Systems – Distributed Systems – Clustered Systems - Real Time Systems – Hardware Protection – System Components – Handheld Systems - Operating System Services – System Calls – System Programs – Process Concept – Process Scheduling – Operations on Processes – Cooperating Processes – Inter-process Communication.

2 Marks

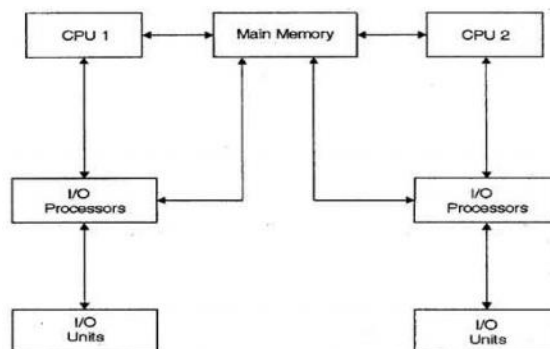
1. What are the five major activities of an operating system with regard to process management? (APR'14)

The various activities that the operating system performs with regard to process management are mainly process scheduling and context switching. The five major activities are:

- The creation and deletion of both user and system processes
- The suspension and resumption of processes
- The provision of mechanisms for process synchronization
- The provision of mechanisms for process communication
- The provision of mechanisms for deadlock handling

2. What are multiprocessor systems & give their advantages? (,APR'14,APR'15)

Multiprocessor Operating System refers to the use of two or more central processing units (CPU) within a single computer system. These multiple CPUs are in a close communication sharing the computer bus, memory and other peripheral devices. These systems are referred as tightly coupled systems.



- Multiprocessor system supports the processes to run in parallel.
- The main advantage of multiprocessor system is to get more work done in a shorter period of time.
- Moreover, multiprocessor systems prove more reliable in the situations of failure of one processor.

3. What is meant by context switch? (NOV'14) (APR'16)

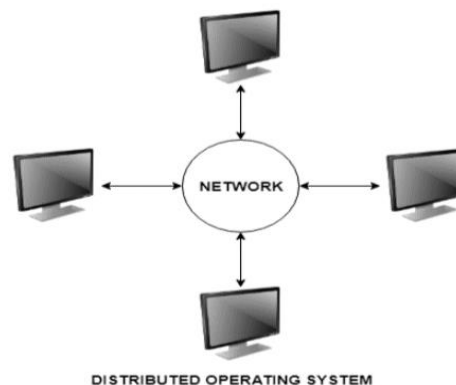
- Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process. This task is known as a Context switch.
- When a context switch occurs, the kernel saves the context of the old process in its PCB and loads the saved context of the new process scheduled to run.
- Context-switch time is pure overhead, because the system does no useful work while switching.

4. What is an Operating system? (APR'15)

- An operating system is a program that manages the computer hardware.
- It also provides a basis for application programs and acts as an intermediary between a user of a computer and the computer hardware.
- The purpose of an operating system is to provide an environment in which a user can execute programs.

5. What is Distributed system? (NOV '15) What is Distributed system? [Nov 2018]

- A distributed system contains multiple nodes that are physically separate but linked together using the network. All the nodes in this system communicate with each other and handle processes in tandem. Each of these nodes contains a small part of the distributed operating system software.



- The nodes in the distributed systems can be arranged in the form of client/server systems or peer to peer systems.

6. Name any three components of the system (Nov '15)

The various system components are,

- Process management
- Main memory management
- File management
- I/O-system management
- Secondary storage management
- Networking

- Protection system
- Command interpreter system

7. Differentiate multiprogramming and multiprocessor operating system. (APR'16)

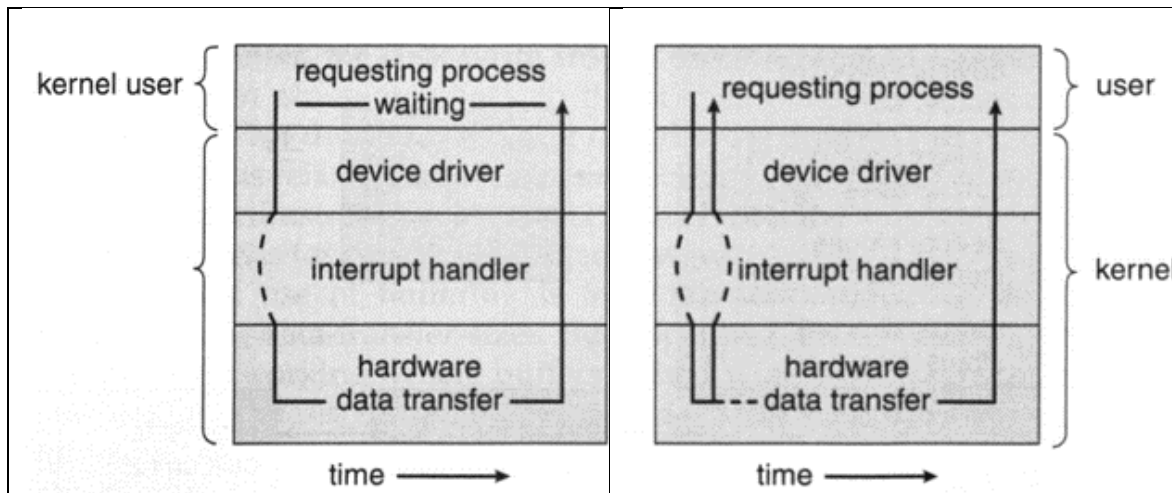
Multiprocessing	Multiprogramming
Multiprocessing refers to processing of multiple processes at same time by multiple CPUs.	Multiprogramming keeps several programs in main memory at the same time and execute them concurrently utilizing single CPU.
It utilizes multiple CPUs.	It utilizes single CPU.
It permits parallel processing.	Context switching takes place.
Less time taken to process the jobs.	More Time taken to process the jobs.
It facilitates much efficient utilization of devices of the computer system.	Less efficient than multiprocessing.
Usually more expensive.	Such systems are less expensive.

8. Why does computer system need operating systems? (Nov 16)

- An operating system is the most important software that runs on a computer.
- It manages the computer's memory and processes, as well as all of its software and hardware.
- It also allows user to communicate with the computer without knowing how to speak the computer's language.
- operating system (OS) manages all of the software and hardware on the computer.
- Most of the time, there are several different computer programs running at the same time, and they all need to access computer's central processing unit (CPU), memory, and storage.
- The operating system coordinates all of this to make sure each program gets what it needs.

9. Differentiate blocking and non-blocking communications. (Nov 16)

Blocking Communication	Non-Blocking communication
It is Synchronous Communication	It is Asynchronous communication
Send Blocks until message is actually sent	Send returns immediately
Receive blocks until message is actually received	Return does not block either



10. Differentiate tightly coupled systems and loosely coupled systems? (May 2017)

Tightly coupled system	Loosely coupled system
It has shared memory concept	It has distributed memory concept
Contention is high in tightly coupled	Contention is low in loosely coupled
It has low scalability	It has high scalability
It has low delay	It has high delay
Throughput is high in tightly coupled	Throughput is low in loosely coupled
Cost of tightly coupled system is high	Cost of loosely coupled system is low
It has dynamic interconnection network	It has static interconnection network
It operates on Single Operating System	It operates on Multiple Operating System

11. What are the different types of multiprocessing? (May 2017)

- **Symmetric multiprocessing (SMP):** In SMP each processor runs an identical copy of the OS & these copies communicate with one another as needed. All processors are peers.
- Examples are Windows NT, Solaris, Digital UNIX, and OS/2 & Linux.
- **Asymmetric multiprocessing:** Each processor is assigned a specific task. A master processor controls the system; the other processors look to the master for instructions or predefined tasks. It defines a master-slave relationship.
- Example: SunOS Version 4.

12. What is the use of job queues, ready queues & device queues? (May 2017)

- Job queue contains the set of all processes in the system
- Ready queue contains the set of all processes residing in main memory and awaiting execution.
- The list of processes waiting for a particular I/O device is kept in the device queue.

13. Define thread [May 2018]

- Threads are called light weight process. If a process is running a word-processor program, a single thread of instructions is being executed.
- This single thread of control allows the process to perform only one task at one time.
- For example, the user could not simultaneously type in characters and run the spell checker within the same process. Many modern operating systems have extended the process concept to allow a process to have multiple threads of execution. They thus allow the process to perform more than one task at a time.

14. What is Semaphore [May 2018] [Sep 2020]

- Semaphore is a synchronization tool that can resolve complex situation of critical section problems.
- A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations:
 - wait and signal.
 - These operations were originally termed P (for wait; from the Dutch *proberen*, to test) and V (for signal; from *verhogen*, to increment).
- The classical definition of wait in pseudocode is:

```
wait(S) {
    while (S <= 0)
        ; // no-op
        S--;
```

The classical definitions of signal in pseudocode is:

```
Signal(S){
    S++;
}
```

15. Mention any one method for inter process communication [Nov 2018]

Interprocess communication (IPC) is set of interfaces, which is usually programmed in order for the programs to communicate between series of processes. This allows running a program concurrently in an Operating System. These are methods in IPC:

1. Pipes (Same Process) –

This allows flow of data in one direction only. Analogous to simplex systems (Keyboard). Data from the output is usually buffered until input process receives it which must have a common origin.

2. Names Pipes (Different Processes) –

This is a pipe with a specific name it can be used in processes that don't have a shared common process origin. E.g. is FIFO where the details written to a pipe is first named.

3. **Message Queuing –**

This allows messages to be passed between processes using either a single queue or several message queue. This is managed by system kernel this messages are co-ordinated using an API.

4. **Semaphores –**

This is used in solving problems associated with synchronization and to avoid race condition. These are integer values which are greater than or equal to 0.

5. **Shared memory –**

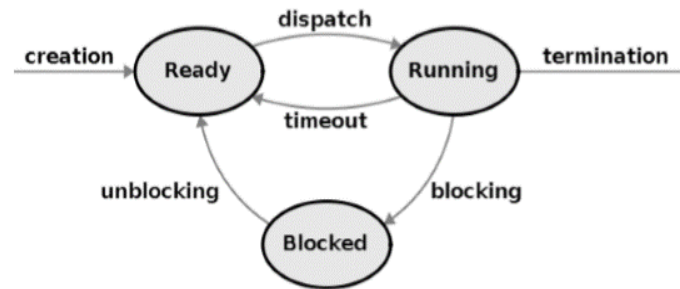
This allows interchange of data through a defined area of a memory. Semaphore values has to be obtained before data can get access to a shared memory.

6. **Sockets –**

This method is mostly used to communicate over a network between a client and a server. It allows for a standard connection which is computer and OS independent.

16. **What is Microkernel [May 2019]**

- A microkernel is a piece of software or even code that contains the near-minimum number of functions and features required to implement an operating system.
- It provides the minimal number of mechanisms, just enough to run the most basic functions of a system, in order to maximize the implementation flexibility so it allows for other parts of the OS to be implemented efficiently since it does not impose a lot of policies.
- The user services are kept in user address space, and kernel services are kept under kernel address space, thus also reduces the size of kernel and size of operating system as well.

17. Draw the process state diagram [May 2019]**18. Why the operating system is viewed as the resource allocator [Nov 2019]**

- A computer system has many resources – hardware & software that may be required to solve a problem, like CPU time, memory space, file-storage space, I/O devices & so on.
- The OS acts as a manager for these resources so it is viewed as a resource allocator.
- The OS is viewed as a control program because it manages the execution of user programs to prevent errors & improper use of the computer.

19. What is the difference between job and process scheduler [Nov 2019]

Job scheduler	Process scheduler
When several jobs are ready to be brought into memory, and there is not enough room for all of them, then the decision made by the system to choose among them can be termed as job scheduling.	The objective of multiprogramming is to have some process running at all times to maximize CPU utilization.
This can also be termed as a long term scheduler	This could be termed as short term scheduler

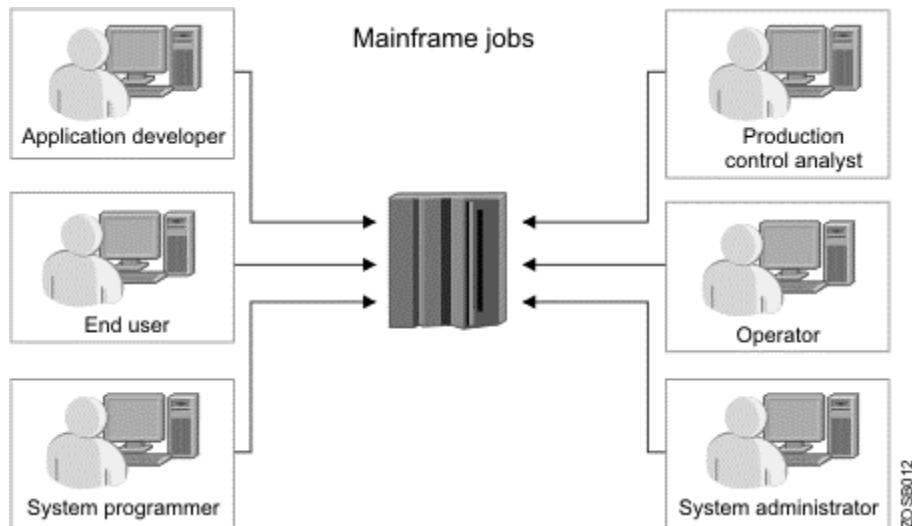
20. What is Throughput? [Sep 2020]

- Throughput is the amount of work completed in a unit of time. In other words, throughput is the processes executed to number of jobs completed in a unit of time.
- The scheduling algorithm must look to maximize the number of jobs processed per time unit.

Part 1 -11 MARKS**1.1 Write in detail about Mainframe systems? (APR'14)**

Mainframes are a type of computer that generally are known for their large size, amount of storage, processing power and high level of reliability. They are primarily used by large organizations for mission-critical applications requiring high volumes of data processing.

Mainframe computer is best platform to execute millions of transactions in every single second. So, mainframe computers are used in the large scale organization, because they have to need for processing the massive data at once in every second.



Components of Mainframe Computer

There are some **major components of mainframe computer** because they play vital role to improve the entire outstanding performance of mainframe computer system.

Processing Unit

CPU has various printed circuits boards, memory modules, different processors, and interfaces for each channels. All channels works as a communication medium in between input/output terminal and memory modules. Main objective of using all those channels is to transfer data and for managing the system components.

Controller Unit

Control unit is also called bus. In mainframe computer system has several buses for different devices such as tape, disk etc, and further it is linked to its storage unit area.

Storage Unit

Storing unit is used to perform different tasks such as insertion data, saving, retrieving, and access data. Storing unit contains the several devices such as hard drives; tape drives, punch cards etc, and these are controlled by CPU. These devices have capacity million time faster to PC.

Multiprocessors

Mainframe computer system consist multiprocessor unit; it means that it has multiple processors for processing the massive data in small time frame with using (Error handling and interrupt handling).

Motherboard

Mainframe's motherboard contains the several high speed processors, main memory (RAM), and different hardware parts which are performed their function through its bus architecture. In this motherboard, 128 bit buses concept is used.

Cluster Controller System

This is special device that is designed to link channel terminal to host terminal system. Cluster controller system has two variants such as Channel-attached cluster controllers and link-attached cluster controllers.

Input/output Channels

Mainframe computer system leads to some techniques like as IOCDS, ESCON, FICON, CHIPD, and more.

IOCDS – IOCDS stands for I/O Control Data Set.

ESCON – ESCON stands for Enterprise Systems Connection.

FICON – FICON stands for Fiber Connector.

Functions of Mainframe Computer

Working of mainframe computer is divided into different segments, below every segment are described.

Data Warehouse System

Every **computer** contains the hard disk for storing the data for long life, but this mainframe computer saves the whole data within itself into application form. When all users try to login from remotely with its connected terminals then mainframe computer allows all remote terminals for accessing their all files as well as programs.

Preserve Authentication Access Permission

Due to store all data and program files into one mainframe system, it can lead to enhance its productivity as well as efficiency. Administrators have access to insert all applications and data into mainframe system, and they can decide how many users should be access for them. So, mainframe system has great firm-wall for harmful intruder's attacks.

Allot Processor Time Frame

Mainframe computer system contains the limited number of processing time to split into all users, who are logged in presently with system. Mainframe system decides that which types of priorities should be linked with different types of users. Administrator has power to select that how to assign processor time.

Examples of Mainframe Computer

- IBM z15
- IBM z14
- Tianhe-1A; NUDT YH Cluster
- Jaguar; Cray XT5
- Nebulae; Dawning TC3600 Blade

Features & Characteristics of Mainframe Computer

- Mainframe computer allows to process huge amount of data simultaneously without getting any malicious attacks.
- Mainframe computers are more popular due to their long-life performance, because it can run smoothly up to 50 years after its proper installation.

- Mainframe application programs get outstanding performance due to their large scaled memory management.
- Mainframe computer systems are capable to share their over workload on the other multiple processors and I/O terminals, and due to this process enhances its performance.
- Mainframe systems have ability to manage different complicated operating systems such as UNIX, VMS, and other IBM O/S like as Z/OS, Z/VM.
- Mainframe systems have less probability in getting any errors, and bugs during processing time. If any time, some errors tries to enter in the system then they are able to remove them.
- Mainframe systems are designed to support “Tightly Coupled Clustering Technology”, due to this feature; it can manage approximate 32 systems along with single system image. If system gets fails due to any **hardware component** damage then running tasks could be shifted to other live system, and any data do not corrupt in this entire process.
- Mainframe system is eligible to support maximum **Input/output devices**.
- Mainframe system is able to execute multiple programs concurrently.
- In mainframe system, virtual storage system can be used.
- It can generate I/O bandwidth in large amount.
- It supports to Zero fault tolerant computing system.
- It is capable to manage several users.
- It can also support centralized computing system.

Advantages of Mainframe Computer

Ultra Computing Power

Mainframe computer is capable to process huge data and run complicated applications with ultra computing speed.

Scale-ability

Mainframe computer can support to extra big power processors, and it can be expended with adding multiple ultra-power processors, memory, and storage device when to require of processing huge data concurrently.

Virtualization System

With using its Virtualization property, Mainframe computer system can be divided into small logical segments for eliminating the memory limitation, and we can great computing performance.

Reliability

Mainframe computer system is able to identify their errors and bugs, and it can self recover of them without any other embedded resources. Today, Modern mainframe computer systems are capable to run frequently for 40 to 50 years without getting any errors.

Self-Serviceability

If, mainframe computer system gets errors during processing time then it is capable to fix them without degrading its performance in short duration.

Protection

Mostly, mainframe computer are used by large scale organizations because they need to secure their confidential data. So, mainframe computer system allows to more attention on authentication protection for storing data.

Flexible Customization

Mainframe computer system's Customization is performed as per the client's requirement. Mainframe computer can support multiple **operating systems** at same time.

Mainframe computer systems are getting more popularity due to their long lasting performance (up to 40 years).

Mainframe computer system can support millions of transactions per second.

It can control the millions of users and applications concurrently.

Disadvantages of Mainframe Computer

- Mainframe systems are very expensive, and these **types of computer** cannot be used in the home. Due to its more processing power, they are used by large scale companies, banks, and other government sectors.
- In mainframe computer system, we can use the normal **operating system** such as Windows and Android. In mainframe system is used the custom operating system and hardware as the client need, and they are more costly.
- Mainframe computer system requires the more space and less temperature.
- If, any errors or bugs are occurred while performing the massive task, then need the well trained staff for eliminating the errors.
- Entire system can gets down due to major damage of **hardware component's system**.
- Difficult to read its instructions which are used on the command based interface.
- It Consume more resources.

1.2 Write about process scheduling (NOV'14)

- Process Scheduling is an OS task that schedules processes of different states like ready, waiting, and running.
- Process scheduling allows OS to allocate a time interval of CPU execution for each process.
- Another important reason for using a process scheduling system is that it keeps the CPU busy all the time. This allows the user to get the minimum response time for programs.
- The prime aim of the process scheduling system is to keep the CPU busy all the time and to deliver minimum response time for all programs. For achieving this, the scheduler must apply appropriate rules for swapping processes **IN** and **OUT** of CPU.

Scheduling fell into one of the two general categories:

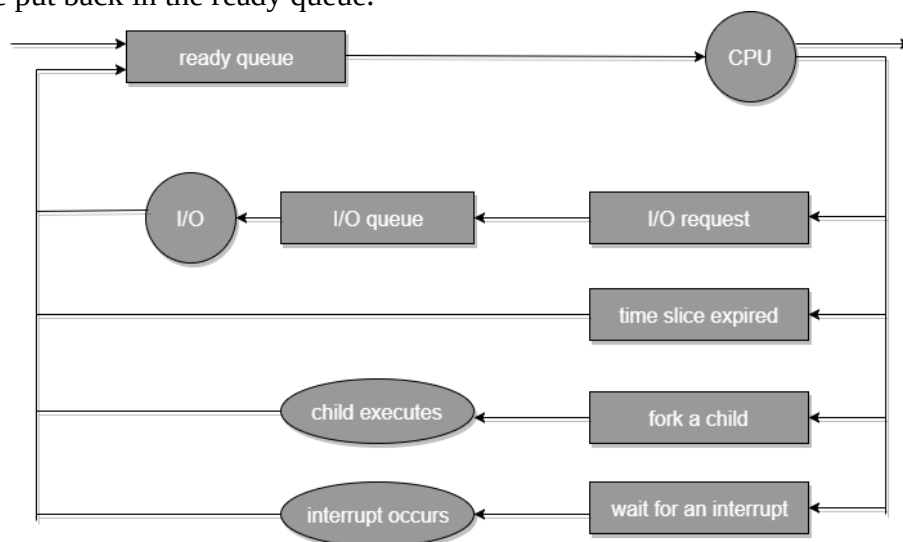
- **Non Pre-emptive Scheduling:** When the currently executing process gives up the CPU voluntarily.
- **Pre-emptive Scheduling:** When the operating system decides to favour another process, pre-empting the currently executing process.

Scheduling Queues

- All processes, upon entering into the system, are stored in the **Job Queue**.
- Processes in the Ready state are placed in the **Ready Queue**.
- Processes waiting for a device to become available are placed in **Device Queues**. There are unique device queues available for each I/O device.

A new process is initially put in the **Ready queue**. It waits in the ready queue until it is selected for execution (or dispatched). Once the process is assigned to the CPU and is executing, one of the following several events can occur:

- The process could issue an I/O request, and then be placed in the **I/O queue**.
- The process could create a new subprocess and wait for its termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.



In the first two cases, the process eventually switches from the waiting state to the ready state, and is then put back in the ready queue. A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.

Types of Schedulers - There are three types of schedulers available:

1. Long Term Scheduler
2. Short Term Scheduler
3. Medium Term Scheduler

Long Term Scheduler

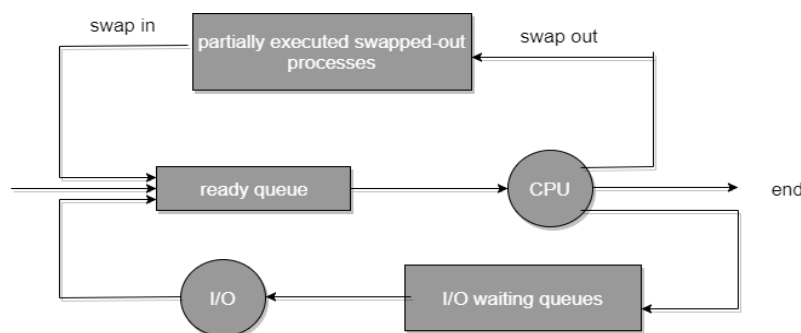
- Long term scheduler runs less frequently.
- Long Term Schedulers decide which program must get into the job queue.
- From the job queue, the Job Processor, selects processes and loads them into the memory for execution.
- Primary aim of the Job Scheduler is to maintain a good degree of Multiprogramming.
- An optimal degree of Multiprogramming means the average rate of process creation is equal to the average departure rate of processes from the execution memory.

Short Term Scheduler

- This is also known as CPU Scheduler and runs very frequently.
- The primary aim of this scheduler is to enhance CPU performance and increase process execution rate.
- A scheduling algorithm is used to select which job is going to be dispatched for the execution. The Job of the short-term scheduler can be very critical in the sense that if it selects job whose CPU burst time is very high then all the jobs after that, will have to wait in the ready queue for a very long time.
- This problem is called starvation which may arise if the short-term scheduler makes some mistakes while selecting the job.

Medium Term Scheduler

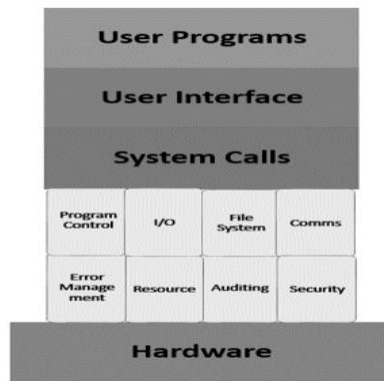
- This scheduler removes the processes from memory (and from active contention for the CPU), and thus reduces the degree of multiprogramming. At some later time, the process can be reintroduced into memory and its execution can be continued where it left off.
- This scheme is called **swapping**. The process is swapped out, and is later swapped in, by the medium term scheduler.
- Swapping may be necessary to improve the process mix, or because a change in memory requirements has overcommitted available memory, requiring memory to be freed up.
- This complete process is described in the below diagram:



1.3 Explain Different type of system calls? (APR'15) [Nov 2018] [Nov 2014] Explain the various types of system calls with an example for each [Nov 2019]

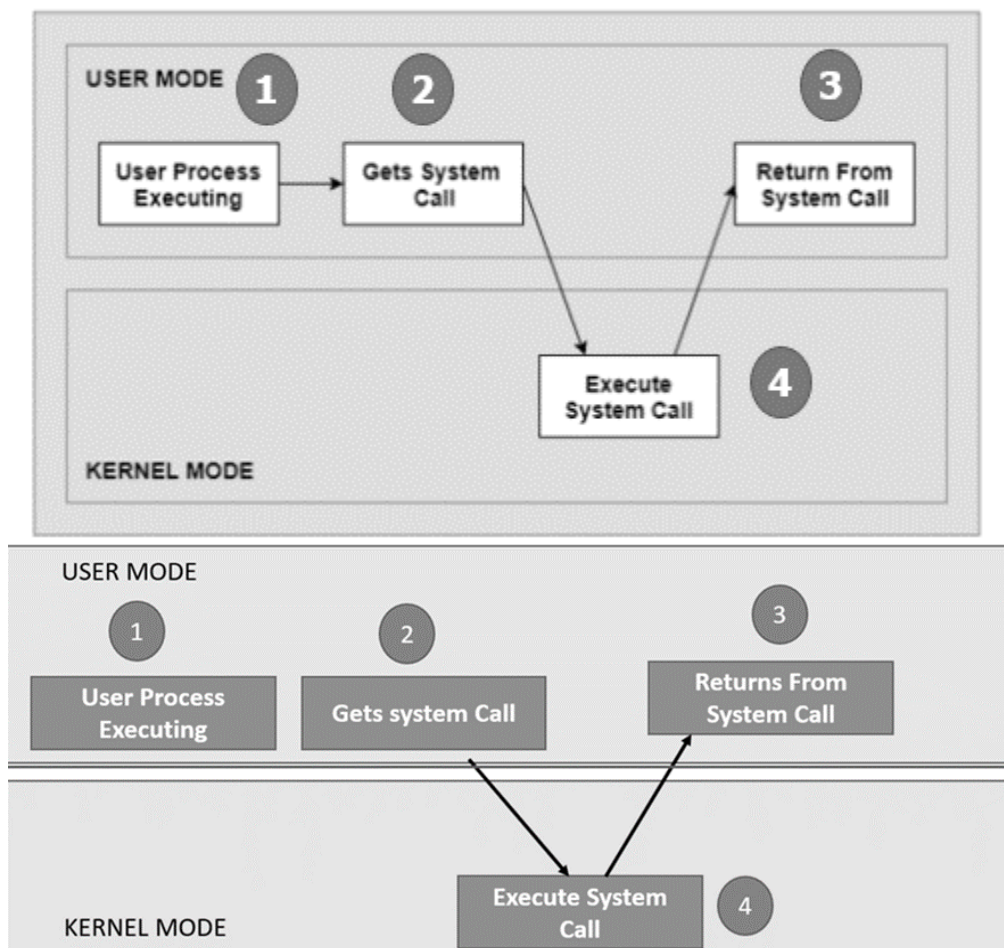
A **system call** is a mechanism that provides the interface between a process and the operating system. It is a programmatic method in which a computer program requests a service from the kernel of the OS.

System call offers the services of the operating system to the user programs via API (Application Programming Interface). System calls are the only entry points for the kernel system.



System Calls in Operating System

Working of System Call



Architecture of the System Call

Step 1) The processes executed in the user mode till the time a system call interrupts it.

Step 2) After that, the system call is executed in the kernel-mode on a priority basis.

Step 3) Once system call execution is over, control returns to the user mode.,

Step 4) The execution of user processes resumed in Kernel mode.

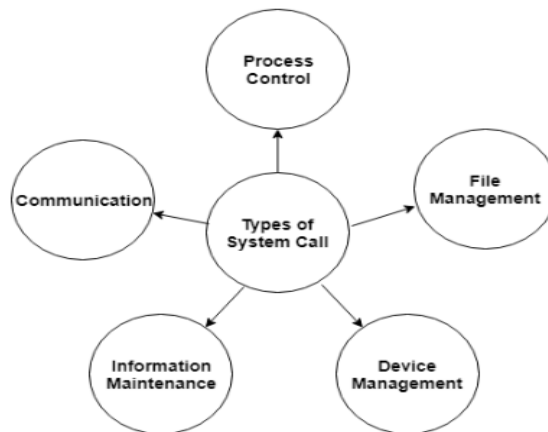
Need of System Calls in OS

Following are situations which need system calls in OS:

- Reading and writing from files demand system calls.
- If a file system wants to create or delete files, system calls are required.
- System calls are used for the creation and management of new processes.
- Network connections need system calls for sending and receiving packets.
- Access to hardware devices like scanner, printer, need a system call.

Types of System calls - Here are the five types of system calls used in OS:

- Process Control
- File Management
- Device Management
- Information Maintenance
- Communications



Process Control - This system calls perform the task of process creation, process termination, etc.

- End and Abort
- Load and Execute
- Create Process and Terminate Process
- Wait and Signed Event
- Allocate and free memory

File Management - File management system calls handle file manipulation jobs like creating a file, reading, and writing, etc.

- Create a file
- Delete file
- Open and close file
- Read, write, and reposition
- Get and set file attributes

Device Management - Device management does the job of device manipulation like reading from device buffers, writing into device buffers, etc.

- Request and release device
- Logically attach/ detach devices
- Get and Set device attributes

Information Maintenance - It handles information and its transfer between the OS and the user program.

- Get or set time and date
- Get process and device attributes

Communication: These types of system calls are specially used for interprocess communications.

- Create, delete communications connections
- Send, receive message
- Help OS to transfer status information
- Attach or detach remote devices

Rules for passing Parameters for System Call

Here are general common rules for passing parameters to the System Call:

- Parameters should be pushed on or popped off the stack by the operating system.
- Parameters can be passed in registers.
- When there are more parameters than registers, it should be stored in a block, and the block address should be passed as a parameter to a register.

Important System Calls Used in OS

wait()

- In some systems, a process needs to wait for another process to complete its execution. This type of situation occurs when a parent process creates a child process, and the execution of the parent process remains suspended until its child process executes.
- The suspension of the parent process automatically occurs with a wait() system call. When the child process ends execution, the control moves back to the parent process.

fork()

- Processes use this system call to create processes that are a copy of themselves.
- With the help of this system Call parent process creates a child process, and the execution of the parent process will be suspended till the child process executes.

exec()

- This system call runs when an executable file in the context of an already running process that replaces the older executable file.
- However, the original process identifier remains as a new process is not built, but stack, data, head, data, etc. are replaced by the new process.

kill():

- The kill() system call is used by OS to send a termination signal to a process that urges the process to exit.
- However, a kill system call does not necessarily mean killing the process and can have various meanings.

exit():

- The exit() system call is used to terminate program execution.
- Specially in the multi-threaded environment, this call defines that the thread execution is complete.
- The OS reclaims resources that were used by the process after the use of exit() system call.

Categories	Windows	Unix
------------	---------	------

Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
Device manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
File manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	Open() Read() write() close()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	Pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup ()	Chmod() Umask() Chown()

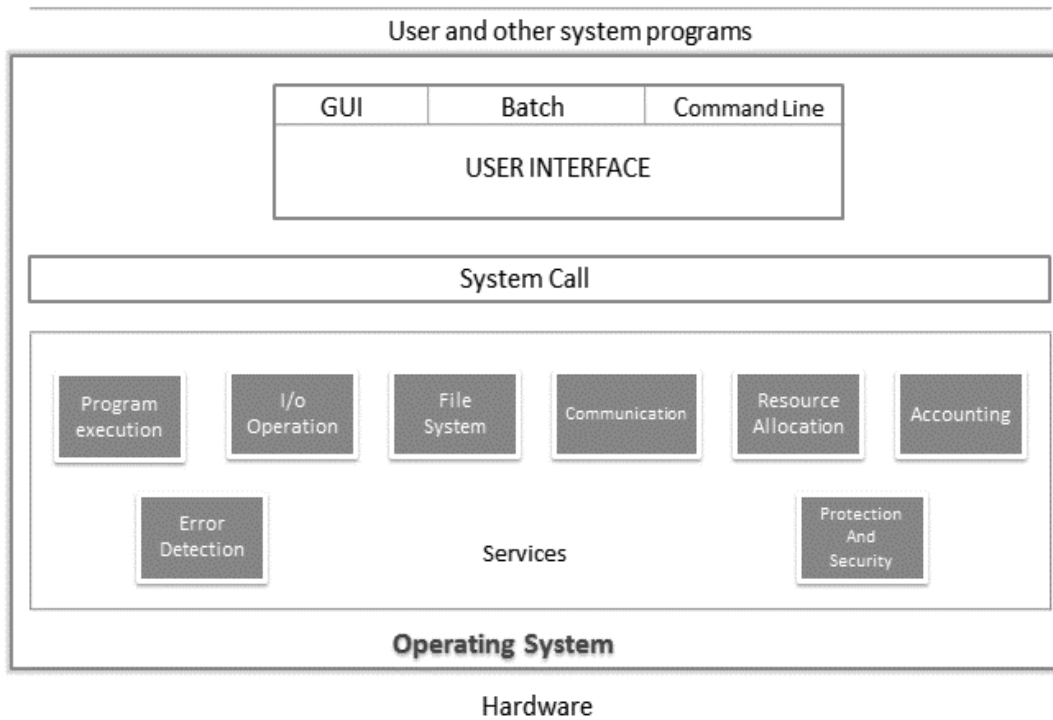
1.4 Write notes on Operating System Services? (NOV '15)

Explain the services provided by the operating systems [2020]

An Operating System supplies different kinds of services to both the users and to the programs as well. It also provides application programs (that run within an Operating system) an environment to execute it freely. It provides users the services run various programs in a convenient manner.

Here is a list of common services offered by an almost all operating systems:

- User Interface
- Program Execution
- File system manipulation
- Input / Output Operations
- Communication
- Resource Allocation
- Error Detection
- Accounting
- Security and protection



User Interface of Operating System

Usually, Operating system comes in three forms or types. Depending on the interface their types have been further subdivided. These are:

- Command line interface
- Batch based interface
- Graphical User Interface
 - The command line interface (CLI) usually deals with using text commands and a technique for entering those commands.
 - The batch interface (BI): commands and directives are used to manage those commands that are entered into files and those files get executed.
 - Another type is the graphical user interface (GUI): which is a window system with a pointing device (like mouse or trackball) to point to the I/O, choose from menus driven interface and to make choices viewing from a number of lists and a keyboard to entry the texts.

Program Execution in Operating System

- The operating system must have the capability to load a program into memory and execute that program.
- Furthermore, the program must be able to end its execution, either normally or abnormally / forcefully.

File System Manipulation in Operating System

- Programs need has to be read and then write them as files and directories.

- File handling portion of operating system also allows users to create and delete files by specific name along with extension, search for a given file and / or list file information.
- Some programs comprise of permissions management for allowing or denying access to files or directories based on file ownership.

I/O operations in Operating System

- A program which is currently executing may require I/O, which may involve file or another I/O device.
- For efficiency and protection, users cannot directly govern the I/O devices.
- So, the OS provide a means to do I/O Input / Output operation which means read or write operation with any file.

Communication System of Operating System

- Process needs to swap over information with other process.
- Processes executing on same computer system or on different computer systems can communicate using operating system support.
- Communication between two processes can be done using shared memory or via message passing.

Resource Allocation of Operating System

- When multiple jobs running concurrently, resources must need to be allocated to each of them.
- Resources can be CPU cycles, main memory storage, file storage and I/O devices. CPU scheduling routines are used here to establish how best the CPU can be used.

Error Detection

- Errors may occur within CPU, memory hardware, I/O devices and in the user program.
- For each type of error, the OS takes adequate action for ensuring correct and consistent computing.

Accounting

- This service of the operating system keeps track of which users are using how much and what kinds of computer resources have been used for accounting or simply to accumulate usage statistics.

Protection and Security

- Protection includes in ensuring all access to system resources in a controlled manner.
- For making a system secure, the user needs to authenticate him or her to the system before using (usually via login ID and password).

1.5 Discuss Co-operating process with an example (APR'16)

Cooperating processes are those that can affect or are affected by other processes running on the system. Cooperating processes may share data with each other.

Reasons for needing cooperating processes

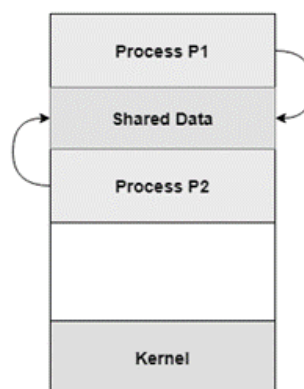
- **Modularity**
Modularity involves dividing complicated tasks into smaller subtasks. These subtasks can complete by different cooperating processes. This leads to faster and more efficient completion of the required tasks.
- **Information Sharing**
Sharing of information between multiple processes can be accomplished using cooperating processes. This may include access to the same files. A mechanism is required so that the processes can access the files in parallel to each other.
- **Convenience**
There are many tasks that a user needs to do such as compiling, printing, editing etc. It is convenient if these tasks can be managed by cooperating processes.
- **Computation Speedup**
Subtasks of a single task can be performed parallelly using cooperating processes. This increases the computation speedup as the task can be executed faster. However, this is only possible if the system has multiple processing elements.

Methods of Cooperation

Cooperating processes can coordinate with each other using shared data or messages.

- **Cooperation by Sharing**
The cooperating processes can cooperate with each other using shared data such as memory, variables, files, databases etc. Critical section is used to provide data integrity and writing is mutually exclusive to prevent inconsistent data.

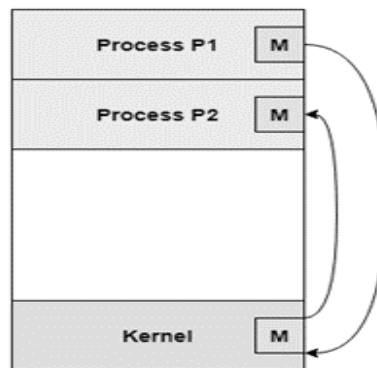
A diagram that demonstrates cooperation by sharing is given as follows –



In the above diagram, Process P1 and P2 can cooperate with each other using shared data such as memory, variables, files, databases etc.

- **Cooperation by Communication**

The cooperating processes can cooperate with each other using messages. This may lead to deadlock if each process is waiting for a message from the other to perform an operation. Starvation is also possible if a process never receives a message. A diagram that demonstrates cooperation by communication is given as follows –



In the above diagram, Process P1 and P2 can cooperate with each other using messages to communicate.

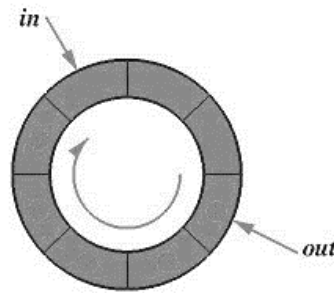
A cooperating process is one that can affect or be affected by other process executing in the system cooperating process an:

1. **Directly share a logical address data space (i.e., code & data)**
2. **Share data only through files/ messages**

Example- producer-consumer problem

- To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer.
- A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced. In this situation, the consumer must wait until an item is produced.
- The *unbounded-buffer producer-consumer* problem places no practical limit on the size of the buffer. The consumer may have to wait for new items, but the producer can always produce new items.
- The *bounded-buffer producer consumer* problem assumes a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full.
- The buffer may either be provided by the operating system through the use of an inter process-communication (IPC) or by explicitly coded by the application programmer with the use of shared memory.

The producer and consumer processes share the following variables:



```
#define BUFFER_SIZE 10
typedef struct {
    . . . } item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int counter = 0;
#define BUFFER-SIZE 10
typedef s t r u c t {
    . . .
} item;
item buffer [BUFFER-SIZE];
i n t i n = 0;
i n t o u t = 0;
```

The shared buffer is implemented as a circular array with two logical pointers: **in** and **out**.
 The variable **in** points to the next free position in the buffer;
 The variable **out** points to the first full position in the buffer.
 The buffer is empty when **in == out**;
 The buffer is full when **((in + 1) % BUFFERSIZE) == out**.

Producer process

The producer process has a local variable **nextproduced** in which the new item to be produced is stored

```
while (1) {
    /* produce an item in nextproduced */
    while ( ((in + 1) % BUFFER-SIZE) == out)
        ; /* do nothing */
    buffer[in] = nextproduced;
    in = (in + 1) % BUFFER-SIZE;
```

Consumer process - The consumer process has a local variable **nextconsumed** in which the item to be consumed is stored

```
while (1) {
    while (in == out)
        ; // do nothing
    nextconsumed = buffer [out];
    out = (out + 1) % BUFFER-SIZE;
    /* consume the item in nextconsumed */
```

Here,

- in variable is used by producer to identify the next empty slot in the buffer.
- out variable is used by the consumer to identify where it has to the consumer the item.
- counter is used by producer and consumer to identify the number of filled slots in the buffer.

Shared Resources

1. **buffer**
2. **counter**

When producer and consumer are not executed can current then inconsistency arises. Here the value of a counter that is used by both producer and consumer will be wrong if both are executed concurrently without any control. The producer and consumer processes share the following variables:

```
var Buffer : array [0,n-1] of item;
in, out:0..n-1;
```

With the variables in and out initialized to the value 0. In The shared buffer there are two logical pointers; in and out that is implemented in the form of a circular array. The in variables points to the next free position in the buffer and out variable points to the first full position in the buffer. When, in = out the buffer is empty and when $in+1 \bmod n = out$ the buffer is full.

1.6 What are the benefits of Inter-Process communication? Explain about how two different processes can communicate with each other using shared memory? (NOV'16)

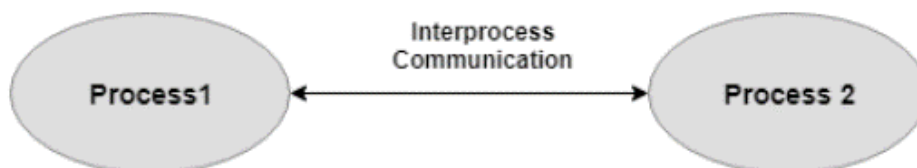
Write about Inter process Communication? (APR' 14, NOV '15)

Discuss how inter-process communication takes place in Operating systems [Nov 2017]

What is IPC? Explain [6] [May 2018]

Describe the PIPEs IPC mechanism [May 2019]

Inter process communication (IPC) is a mechanism which allows processes to communicate with each other and synchronize their actions. The communication between these processes can be seen as a method of co-operation between them.



Processes can communicate with each other through both:

- Shared Memory

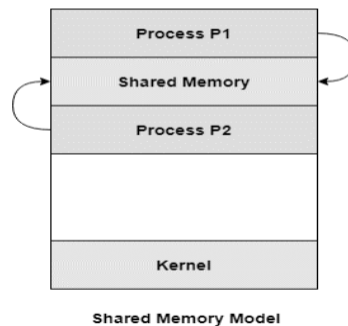
- Message passing

An operating system can implement both method of communication.

Shared Memory Model of Process Communication

The shared memory in the shared memory model is the memory that can be simultaneously accessed by multiple processes. This is done so that the processes can communicate with each other. All POSIX systems, as well as Windows operating systems use shared memory.

A diagram that illustrates the shared memory model of process communication is given as follows –



In the above diagram, the shared memory can be accessed by Process 1 and Process 2.

Advantage of Shared Memory Model

Memory communication is faster on the shared memory model as compared to the message passing model on the same machine.

Disadvantage of Shared Memory Model

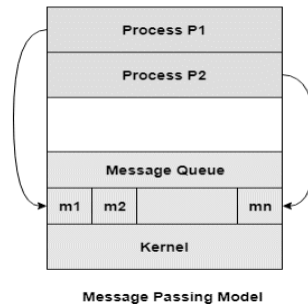
Some of the disadvantages of shared memory model are as follows –

- All the processes that use the shared memory model need to make sure that they are not writing to the same memory location.
- Shared memory model may create problems such as synchronization and memory protection that need to be addressed.

Message Passing Model of Process Communication

Message passing model allows multiple processes to read and write data to the message queue without being connected to each other. Messages are stored on the queue until their recipient retrieves them. Message queues are quite useful for interprocess communication and are used by most operating systems.

A diagram that demonstrates message passing model of process communication is given as follows –



In the above diagram, both the processes P1 and P2 can access the message queue and store and retrieve data.

Advantages of Message Passing Model

Some of the advantages of message passing model are given as follows –

- The message passing model is much easier to implement than the shared memory model.
- It is easier to build parallel hardware using message passing model as it is quite tolerant of higher communication latencies.

Disadvantage of Message Passing Model

The message passing model has slower communication than the shared memory model because the connection setup takes time.

Synchronization in Interprocess Communication

Synchronization is a necessary part of interprocess communication. It is either provided by the interprocess control mechanism or handled by the communicating processes.

Some of the methods to provide synchronization are as follows –

- **Semaphore**
A semaphore is a variable that controls the access to a common resource by multiple processes. The two types of semaphores are binary semaphores and counting semaphores.
- **Mutual Exclusion**
Mutual exclusion requires that only one process thread can enter the critical section at a time. This is useful for synchronization and also prevents race conditions.
- **Barrier**
A barrier does not allow individual processes to proceed until all the processes reach it. Many parallel languages and collective routines impose barriers.
- **Spinlock**
This is a type of lock. The processes trying to acquire this lock wait in a loop while checking if the lock is available or not. This is known as busy waiting because the process is not doing any useful operation even though it is active.

Approaches to Interprocess Communication

The different approaches to implement interprocess communication are given as follows –

- **Pipe**

A pipe is a data channel that is unidirectional. Two pipes can be used to create a two-way data channel between two processes. This uses standard input and output methods. Pipes are used in all POSIX systems as well as Windows operating systems.

- **Socket**

The socket is the endpoint for sending or receiving data in a network. This is true for data sent between processes on the same computer or data sent between different computers on the same network. Most of the operating systems use sockets for interprocess communication.

- **File**

A file is a data record that may be stored on a disk or acquired on demand by a file server. Multiple processes can access a file as required. All operating systems use files for data storage.

- **Signal**

Signals are useful in interprocess communication in a limited way. They are system messages that are sent from one process to another. Normally, signals are not used to transfer data but are used for remote commands between processes.

- **Shared Memory**

Shared memory is the memory that can be simultaneously accessed by multiple processes. This is done so that the processes can communicate with each other. All POSIX systems, as well as Windows operating systems use shared memory.

- **Message Queue**

Multiple processes can read and write data to the message queue without being connected to each other. Messages are stored in the queue until their recipient retrieves them. Message queues are quite useful for interprocess communication and are used by most operating systems.

Interprocess Communication with Sockets

One of the ways to manage interprocess communication is by using sockets. They provide point-to-point, two-way communication between two processes. Sockets are an endpoint of communication and a name can be bound to them. A socket can be associated with one or more processes.

Types of Sockets

- **Sequential Packet Socket:** This type of socket provides a reliable connection for datagrams whose maximum length is fixed. This connection is two-way as well as sequenced.
- **Datagram Socket:** A two-way flow of messages is supported by the datagram socket. The receiver in a datagram socket may receive messages in a different order than that in which they were sent. The operation of datagram sockets is similar to that of passing letters from the source to the destination through a mail.
- **Stream Socket:** Stream sockets operate like a telephone conversation and provide a two-way and reliable flow of data with no record boundaries. This data flow is also sequenced and unduplicated.
- **Raw Socket:** The underlying communication protocols can be accessed using the raw sockets.

Socket Creation

Sockets can be created in a specific domain and the specific type using the following declaration – `int socket(int domain,int type,int protocol)`

If the protocol is not specified in the above system call, the system uses a default protocol that supports the socket type. The socket handle is returned. It is a descriptor.

The bind function call is used to bind an internet address or path to a socket. This is shown as follows – `int bind(int s,const struct sockaddr*name,int namelen)`

Connecting Stream Sockets

Connecting the stream sockets is not a symmetric process. One of the processes acts as a server and the other acts as a client.

The server specifies the number of connection requests that can be queued using the following declaration – `int listen(int s,int backlog)`

The client initiates a connection to the server's socket by using the following declaration – `int connect(int s,struct sockaddr*name,int namelen)`

A new socket descriptor which is valid for that particular connection is returned by the following declaration – `int accept(int s,struct sockaddr*addr,int*addrlen)`

Stream Data Transfer

The `send()` and `recv()` functions are used to send and receive data using sockets. These are similar to the `read()` and `write()` functions but contain some extra flags. The declaration for `send()` and `recv()` are as follows –

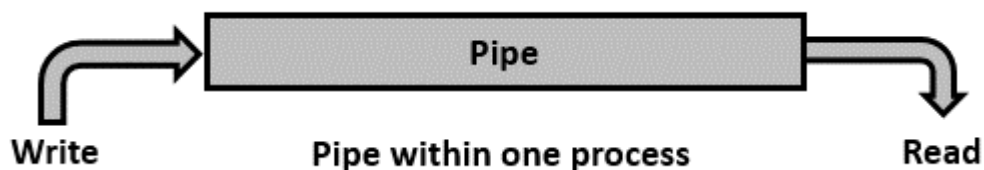
`int send(int s,const char*msg,intlen,int flags)`
`int recv(int s,char*buf,intlen,int flags)`

Stream Closing

The socket is discarded or closed by calling `close()`.

Inter Process Communication - Pipes

A **Pipe** is a technique used for inter process communication. A pipe is a mechanism by which the output of one process is directed into the input of another process. Thus it provides one way flow of data between two related processes.



Although pipe can be accessed like an **ordinary file**, the system actually manages it as **FIFO queue**. A pipe file is created using the pipe system call. A pipe has an input end and an output end.

One can write into a pipe from input end and read from the output end. A pipe descriptor, has an array that stores two pointers, one pointer is for its input end and the other pointer is for its output end.

Suppose **two processes**, Process A and Process B, need to communicate. In such a case, it is important that the process which writes, closes its read end of the pipe and the process which reads, closes its write end of a pipe.

Essentially, for a communication from Process A to Process B the following should happen.

- Process A should keep its write end open and close the read end of the pipe.
- Process B should keep its read end open and close its write end. When a pipe is created, it is given a fixed size in bytes.

When a process attempts to write into the pipe, the write request is immediately executed if the pipe is not full.

However, if pipe is full the process is blocked until the state of pipe changes. Similarly, a reading process is blocked, if it attempts to read more bytes that are currently in pipe, otherwise the reading process is executed. Only one process can access a pipe at a time.

Limitations :

- As a channel of communication a pipe operates in one direction only.
- Pipes cannot support broadcast i.e. sending message to multiple processes at the same time.
- The read end of a pipe reads any way. It does not matter which process is connected to the write end of the pipe. Therefore, this is very insecure mode of communication.
- Some plumbing (closing of ends) is required to create a properly directed pipe.

Advantages of Inter Process Communication (IPC)

- Information sharing
- Modularity/Convenience
- Use of shared memory for communication, limits Remote Procedure Call communication on the local machine
- Only users with access to the shared memory can view the calls

1.8 Explain the differences between OS for mainframe computers and personal computers [6] [May 2018]

Mainframe systems OS	Personal computers OS
Generally, operating systems for Mainframe have simpler requirements than for personal computers. Mainframe systems do not have to be concerned with	An operating system for a PC must be concerned with response time for an interactive user

interacting with a user as much as a personal computer.	
A pure Mainframe system also may have not to handle time sharing,	An operating system for PC must switch rapidly between different jobs
mainframe OS can be used by many users at the same time so it must need to service for many users	a personal computer operating system designed for one user normally
A mainframe OS is more powerful and expensive than pc OS	PC OS is not that much powerful like Mainframes. Hence it is Least expensive
Mainframe OS designed to huge process from many users and it means, it manages lots of I/O for many users	In pc OS there is just one user to log in it means, it does not manages lots of I/O for one user.
OS for a main frame is targeted to handle hundreds of users at a time.	the PC operating systems are not truly multiuser.
The mainframe computers can run more than one operating system on one machine	It is certainly an impossible task for any personal computer to carry
IBM Z series, Unisys Libra	Windows, OS X, Amiga OS, and Linux.

1.9 Why are distributed systems desirable [5] [May 2018]?

A distributed system is a collection of independent computers that appears to its users as a single coherent system.

All the computers are tied together in a network either a Local Area Network (LAN) or Wide Area Network (WAN), communicating with each other so that different portions of a Distributed application run on different computers from any geographical location.

Distributed systems are characterized by their structure: a typical distributed system will consist of some large number of interacting devices that each run their own programs but that are affected by receiving messages or observing shared-memory updates or the states of other devices.

Examples of distributed systems range from simple systems in which a single client talks to a single server to huge amorphous networks like the Internet as a whole.

Desirable Features

Transparency

- Object class level
- System call and interprocess communication level

Minimal interference

- Can be done by minimizing freezing time
- Freezing time: a time for which the execution of the process is stopped for transferring its information to the destination node

Minimal residual dependencies

- Migrated process should not continue to depend on its previous node once it has started executing on new node

Efficiency

- Time required of migrating a process
- The cost of locating an object
- The cost of supporting remote execution once the process is migrated

Robustness

- The failure of a node other than the one on which a process is currently running should not affect the execution of that process

Communication between co-processes of a job

1.10 Write notes on [Nov 2018]

- a. Multi-processor systems
- b. Real time systems

Explain different types of systems [Sep 2020]

Differentiate between Mutliprocessor and Distributed systems [Nov 2019]

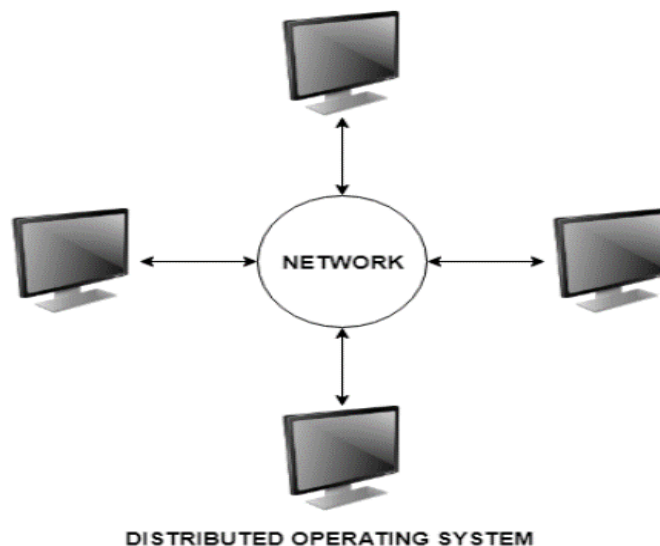
What are the advantages and disadvantages of multiprocessors system? (6) (May 2017)

Describe the differences between symmetric and asymmetric multiprocessing. What are the advantages and disadvantages of multiprocessors system? (6) (May 2017)

Write short notes on Clustered Systems and real time &hand-held systems? [7] (May 2017)

Distributed Systems

A distributed system contains multiple nodes that are physically separate but linked together using the network. All the nodes in this system communicate with each other and handle processes in tandem. Each of these nodes contains a small part of the distributed operating system software.



Types of Distributed Systems

The nodes in the distributed systems can be arranged in the form of client/server systems or peer to peer systems. Details about these are as follows –

Client/Server Systems

In client server systems, the client requests a resource and the server provides that resource. A server may serve multiple clients at the same time while a client is in contact with only one server. Both the client and server usually communicate via a computer network and so they are a part of distributed systems.

Peer to Peer Systems

The peer to peer systems contains nodes that are equal participants in data sharing. All the tasks are equally divided between all the nodes. The nodes interact with each other as required as share resources. This is done with the help of a network.

Advantages of Distributed Computing Systems

- **Reliability, high fault tolerance:** A system crash on one server does not affect other servers.
- **Scalability:** In distributed computing systems you can add more machines as needed.
- **Flexibility:** It makes it easy to install, implement and debug new services.
- **Fast calculation speed:** A distributed computer system can have the computing power of multiple computers, making it faster than other systems.
- **Openness:** Since it is an open system, it can be accessed both locally and remotely.
- **High performance:** Compared to centralized computer network clusters, it can provide higher performance and better cost performance.

Disadvantages of Distributed Computing Systems

- **Difficult troubleshooting:** Troubleshooting and diagnostics are more difficult due to distribution across multiple servers.
- **Less software support:** Less software support is a major drawback of distributed computer systems.
- **High network infrastructure costs:** Network basic setup issues, including transmission, high load, and loss of information.
- **Security issues:** The characteristics of open systems make data security and sharing risks in distributed computer systems.

Examples of distributed systems and applications

1: Telecommunication networks:

- telephone networks and cellular networks
- computer networks such as the Internet
- wireless sensor networks
- routing algorithms

2: Network Applications:

- World Wide Web and peer-to-peer networks
- massively multiplayer online games and virtual reality communities
- distributed databases and distributed database management systems
- network file systems

- distributed cache such as burst buffers
- distributed information processing systems such as banking systems and airline reservation systems

3: Real-Time Process Control:

- aircraft control systems
- industrial control systems

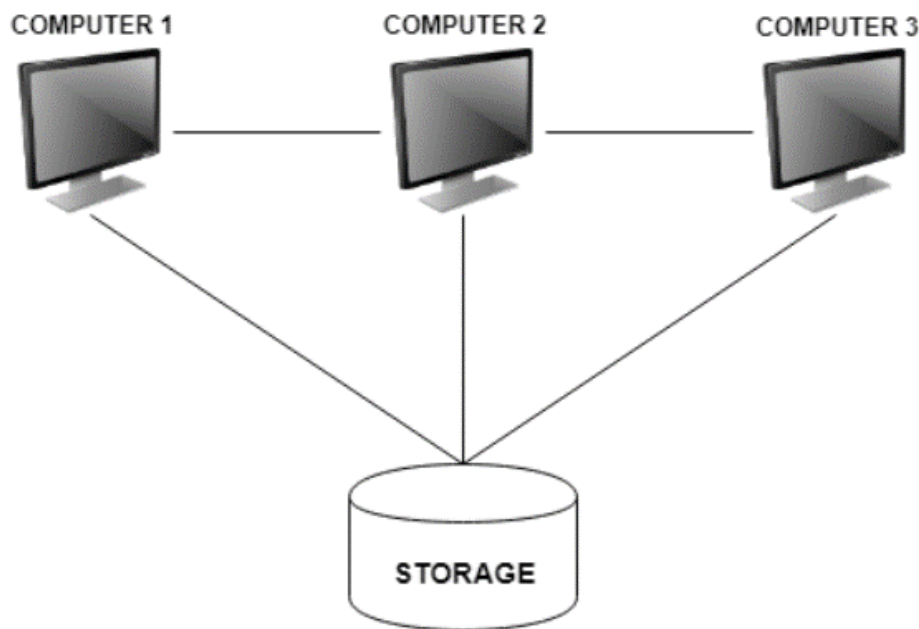
4: Parallel Computation:

- scientific computing, including cluster computing, grid computing, cloud computing, and various volunteer computing projects
- distributed rendering in computer graphics

Clustered systems

Clustered systems are similar to parallel systems as they both have multiple CPUs. However, a major difference is that clustered systems are created by two or more individual computer systems merged together. Basically, they have independent computer systems with a common storage and the systems work together.

A diagram to better illustrate this is –



The clustered systems are a combination of hardware clusters and software clusters. The hardware clusters help in sharing of high-performance disks between the systems. The software clusters make all the systems work together.

Each node in the clustered systems contains the cluster software. This software monitors the cluster system and makes sure it is working as required. If any one of the nodes in the clustered system fail, then the rest of the nodes take control of its storage and resources and try to restart.

Types of Clustered Systems

There are primarily two types of clustered systems i.e. asymmetric clustering system and symmetric clustering system. Details about these are given as follows –

Asymmetric Clustering System

In this system, one of the nodes in the clustered system is in hot standby mode and all the others run the required applications. The hot standby mode is a failsafe in which a hot standby node is part of the system. The hot standby node continuously monitors the server and if it fails, the hot standby node takes its place.

Symmetric Clustering System

In symmetric clustering system two or more nodes all run applications as well as monitor each other. This is more efficient than asymmetric system as it uses all the hardware and doesn't keep a node merely as a hot standby.

Attributes of Clustered Systems

There are many different purposes that a clustered system can be used for. Some of these can be scientific calculations, web support etc. The clustering systems that embody some major attributes are –

- **Load Balancing Clusters**

In this type of clusters, the nodes in the system share the workload to provide a better performance. For example: A web based cluster may assign different web queries to different nodes so that the system performance is optimized. Some clustered systems use a round robin mechanism to assign requests to different nodes in the system.

- **High Availability Clusters**

These clusters improve the availability of the clustered system. They have extra nodes which are only used if some of the system components fail. So, high availability clusters remove single points of failure i.e. nodes whose failure leads to the failure of the system. These types of clusters are also known as failover clusters or HA clusters.

Benefits of Clustered Systems

The different benefits of clustered systems are as follows –

- **Performance**

Clustered systems result in high performance as they contain two or more individual computer systems merged together. These work as a parallel unit and result in much better performance for the system.

- **Fault Tolerance**

Clustered systems are quite fault tolerant and the loss of one node does not result in the loss of the system. They may even contain one or more nodes in hot standby mode which allows them to take the place of failed nodes.

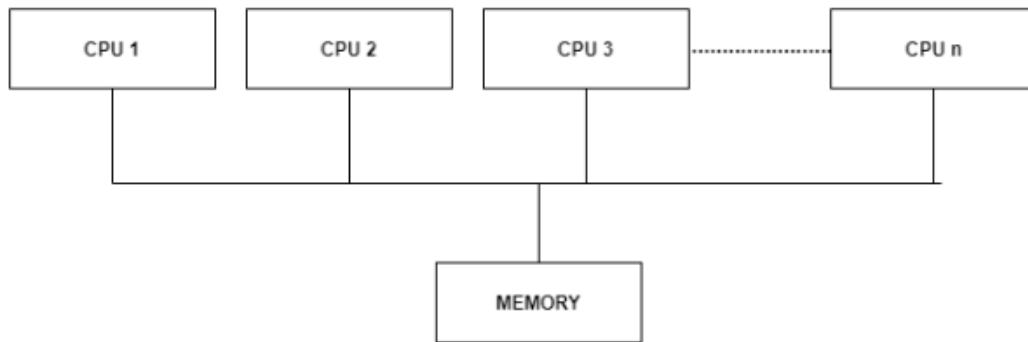
- **Scalability**

Clustered systems are quite scalable as it is easy to add a new node to the system. There is no need to take the entire cluster down to add a new node.

Multiprocessor Systems

Most computer systems are single processor systems i.e they only have one processor. However, multiprocessor or parallel systems are increasing in importance nowadays. These systems have multiple processors working in parallel that share the computer clock,

memory, bus, peripheral devices etc. An image demonstrating the multiprocessor architecture is



Multiprocessing Architecture

Types of Multiprocessors

There are mainly two types of multiprocessors i.e. symmetric and asymmetric multiprocessors. Details about them are as follows –

Symmetric Multiprocessors

In these types of systems, each processor contains a similar copy of the operating system and they all communicate with each other. All the processors are in a peer to peer relationship i.e. no master - slave relationship exists between them.

An example of the symmetric multiprocessing system is the Encore version of Unix for the Multimax Computer.

Asymmetric Multiprocessors

In asymmetric systems, each processor is given a predefined task. There is a master processor that gives instruction to all the other processors. Asymmetric multiprocessor system contains a master slave relationship.

Asymmetric multiprocessor was the only type of multiprocessor available before symmetric multiprocessors were created. Now also, this is the cheaper option.

Advantages of Multiprocessor Systems

More reliable Systems

In a multiprocessor system, even if one processor fails, the system will not halt. This ability to continue working despite hardware failure is known as graceful degradation. For example: If there are 5 processors in a multiprocessor system and one of them fails, then also 4 processors are still working. So the system only becomes slower and does not ground to a halt.

Enhanced Throughput

If multiple processors are working in tandem, then the throughput of the system increases i.e. number of processes getting executed per unit of time increase. If there are N processors then the throughput increases by an amount just under N.

More Economic Systems

Multiprocessor systems are cheaper than single processor systems in the long run because they share the data storage, peripheral devices, power supplies etc. If there are multiple processes that share data, it is better to schedule them on multiprocessor systems with shared data than have different computer systems with multiple copies of the data.

Disadvantages of Multiprocessor Systems**Increased Expense**

Even though multiprocessor systems are cheaper in the long run than using multiple computer systems, still they are quite expensive. It is much cheaper to buy a simple single processor system than a multiprocessor system.

Complicated Operating System Required

There are multiple processors in a multiprocessor system that share peripherals, memory etc. So, it is much more complicated to schedule processes and impart resources to processes than in single processor systems. Hence, a more complex and complicated operating system is required in multiprocessor systems.

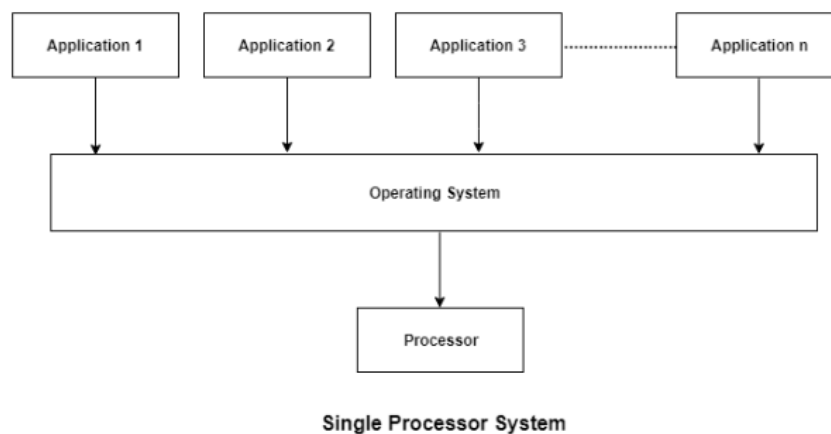
Large Main Memory Required

All the processors in the multiprocessor system share the memory. So a much larger pool of memory is required as compared to single processor systems.

SINGLE PROCESSOR SYSTEM

A single processor system contains only one processor. So only one process can be executed at a time and then the process is selected from the ready queue. Most general-purpose computers contain the single processor systems as they are commonly in use.

A single processor system can be further described using the diagram below –



As in the above diagram, there are multiple applications that need to be executed. However, the system contains a single processor and only one process can be executed at a time.

Most systems use a single processor. In a single-processor system, there is one main CPU capable of executing a general-purpose instruction set, including instructions from user processes.

Almost all systems have other special-purpose processors as well. They may come in the form of device-specific processors, such as disk, keyboard, and graphics controllers; or, on mainframes, they may come in the form of more general-purpose processors, such as I/O processors that move data rapidly among the components of the system.

All of these special-purpose processors run a limited instruction set and do not run user processes. Sometimes they are managed by the operating system, in that the operating system sends them information about their next task and monitors their status.

REAL TIME SYSTEMS

A real-time system is defined as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment. The time taken by the system to respond to an input and display of required updated information is termed as the **response time**. So in this method, the response time is very less as compared to online processing.

Real-time systems are used when there are rigid time requirements on the operation of a processor or the flow of data and real-time systems can be used as a control device in a dedicated application. A real-time operating system must have well-defined, fixed time constraints, otherwise the system will fail. For example, Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

There are two types of real-time operating systems.

Hard real-time systems

Hard real-time systems guarantee that critical tasks complete on time. In hard real-time systems, secondary storage is limited or missing and the data is stored in ROM. In these systems, virtual memory is almost never found.

Soft real-time systems

Soft real-time systems are less restrictive. A critical real-time task gets priority over other tasks and retains the priority until it completes. Soft real-time systems have limited utility than hard real-time systems. For example, multimedia, virtual reality, Advanced Scientific Projects like undersea exploration and planetary rovers, etc.

Reference model of real time system: Our reference model is characterized by three elements:

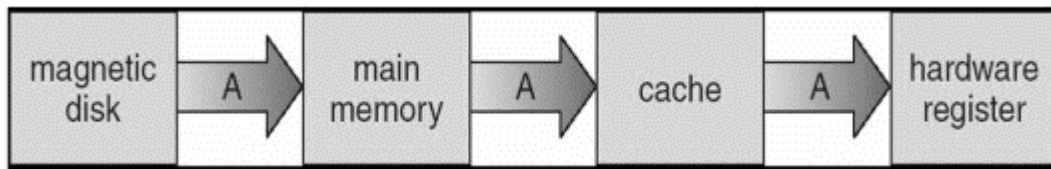
1. **A workload model:** It specifies the application supported by system.
2. **A resource model:** It specifies the resources available to the application.
3. **Algorithms:** It specifies how the application system will use resources.

1.13 Discuss with examples, how the problem of maintaining coherence of cached data manifests itself in the following processing environments: [May 2019]

1. **Single-processor systems**
2. **Multiprocessor systems**

3. Distributed systems

- Suppose an integer A which is to be incremented by 1 is located in file B, and file B resides in magnetic disk.
- The increment operation proceeds by first issuing an I/O operation to copy the disk block on which A resides to main memory. This operation is followed by copying A to the cache and to an internal register.
- Thus, the copy of A appears in several places; on the magnetic disk, in main memory, in the cache and in an internal register
- Once the increment takes place in the internal register, the value of A differs in the various storage systems. The value of A becomes the same only after the new value of A is written from the internal register back to the magnetic disk.



Single processor systems

- In a computing environment where only one process executes at a time, this arrangement poses no difficulties, since access to integer A will always be to the copy at the highest level of hierarchy.
- However, in a multitasking environment, where the CPU is switched back and forth among various processes, extreme care must be taken to ensure that, if several processes wish to access A, then each process will obtain the most recently updated value of A.

Multi-processor systems

- It is a bit more complicated since each of the CPUs might contain its own local cache. In such an environment, a copy of A may exist simultaneously in several caches.
- Since the various CPUs can all execute concurrently, we must make sure that an update to the value of A in one cache is immediately reflected in all other caches where A resides.
- This situation is called cache coherency, and is usually a hardware problem.

Distributed systems

- The situation here is even more complex. In this environment, several copies of the same file can be kept on different computers that are distributed in space.
- Since the various replicas may be accessed and updated concurrently, some distributed systems ensure that, when a replica is updated in one place, all other replicas are brought up to date as soon as possible.

Part 2

2.1 List the responsibilities of operating system in connection with process and memory management (4) [May 2017]

The operating system manages the Primary Memory or Main Memory. Main memory is made up of a large array of bytes or words where each byte or word is assigned a certain address. Main memory is a fast storage and it can be accessed directly by the CPU. For a program to be executed, it should be first loaded in the main memory.

The operating system is responsible for the following activities in connection with memory management.

- Keep track of which parts of memory are currently being used and by whom.
- Decide which processes are to be loaded into memory when memory space becomes available.
- In a multiprogramming system, the OS takes a decision about which process will get Memory and how much.
- Allocates the memory when a process requests
- It also de-allocates the Memory when a process no longer requires or has been terminated.

2.2 Explain the scheduling algorithms in Operating systems [Nov 2017]

There are mainly six types of process scheduling algorithms

1. First Come First Serve (FCFS)
2. Shortest-Job-First (SJF) Scheduling
3. Shortest Remaining Time
4. Priority Scheduling
5. Round Robin Scheduling
6. Multilevel Queue Scheduling

First Come First Serve

First Come First Serve is the full form of FCFS. It is the easiest and most simple CPU scheduling algorithm. In this type of algorithm, the process which requests the CPU gets the CPU allocation first. This scheduling method can be managed with a FIFO queue.

As the process enters the ready queue, its PCB (Process Control Block) is linked with the tail of the queue. So, when CPU becomes free, it should be assigned to the process at the beginning of the queue.

Characteristics of FCFS method:

- It offers non-preemptive and pre-emptive scheduling algorithm.
- Jobs are always executed on a first-come, first-serve basis
- It is easy to implement and use.
- However, this method is poor in performance, and the general wait time is quite high.

Shortest Remaining Time

The full form of SRT is Shortest remaining time. It is also known as SJF preemptive scheduling. In this method, the process will be allocated to the task, which is closest to its completion. This method prevents a newer ready state process from holding the completion of an older process.

Characteristics of SRT scheduling method:

- This method is mostly applied in batch environments where short jobs are required to be given preference.
- This is not an ideal method to implement it in a shared system where the required CPU time is unknown.
- Associate with each process as the length of its next CPU burst. So that operating system uses these lengths, which helps to schedule the process with the shortest possible time.

Priority Based Scheduling

Priority scheduling is a method of scheduling processes based on priority. In this method, the scheduler selects the tasks to work as per the priority.

Priority scheduling also helps OS to involve priority assignments. The processes with higher priority should be carried out first, whereas jobs with equal priorities are carried out on a round-robin or FCFS basis. Priority can be decided based on memory requirements, time requirements, etc.

Round-Robin Scheduling

Round robin is the oldest, simplest scheduling algorithm. The name of this algorithm comes from the round-robin principle, where each person gets an equal share of something in turn. It is mostly used for scheduling algorithms in multitasking. This algorithm method helps for starvation free execution of processes.

Characteristics of Round-Robin Scheduling

- Round robin is a hybrid model which is clock-driven
- Time slice should be minimum, which is assigned for a specific task to be processed. However, it may vary for different processes.
- It is a real time system which responds to the event within a specific time limit.

Shortest Job First

SJF is a full form of (Shortest job first) is a scheduling algorithm in which the process with the shortest execution time should be selected for execution next. This scheduling method can be preemptive or non-preemptive. It significantly reduces the average waiting time for other processes awaiting execution.

Characteristics of SJF Scheduling

- It is associated with each job as a unit of time to complete.
- In this method, when the CPU is available, the next process or job with the shortest completion time will be executed first.
- It is Implemented with non-preemptive policy.
- This algorithm method is useful for batch-type processing, where waiting for jobs to complete is not critical.
- It improves job output by offering shorter jobs, which should be executed first, which mostly have a shorter turnaround time.

Multiple-Level Queues Scheduling

This algorithm separates the ready queue into various separate queues. In this method, processes are assigned to a queue based on a specific property of the process, like the process priority, size of the memory, etc.

However, this is not an independent scheduling OS algorithm as it needs to use other types of algorithms in order to schedule the jobs.

Characteristic of Multiple-Level Queues Scheduling:

- Multiple queues should be maintained for processes with some characteristics.
- Every queue may have its separate scheduling algorithms.
- Priorities are given for each queue.

The Purpose of a Scheduling algorithm

Here are the reasons for using a scheduling algorithm:

- The CPU uses scheduling to improve its efficiency.
- It helps you to allocate resources among competing processes.
- The maximum utilization of CPU can be obtained with multi-programming.
- The processes which are to be executed are in ready queue.

2.3 Write about operation on Processes?[6] (NOV'14, NOV '15)

Discuss about process states and process control block [Nov 2018]

Explain the possible operations that could be applied on process in detail [5] [May 2018]

Process Concept

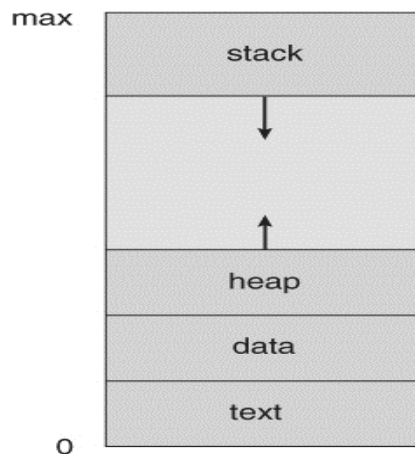
- A process is an instance of a program in execution.
- Batch systems work in terms of "jobs". Many modern process concepts are still expressed in terms of jobs, (e.g. job scheduling), and the two terms are often used interchangeably.

The Process

Process is the execution of a program that performs the actions specified in that program.

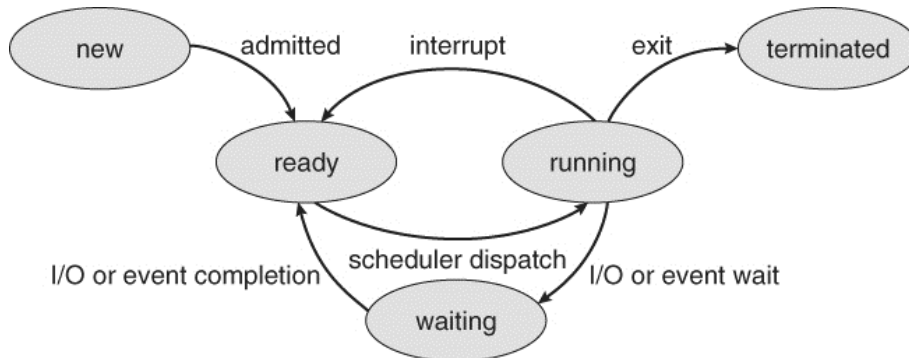
It can be defined as an execution unit where a program runs.

- Process memory is divided into four sections as shown below:
 - The **text section** comprises the compiled program code, read in from non-volatile storage when the program is launched.
 - The **data section** stores global and static variables, allocated and initialized prior to executing main.
 - The **heap** is used for dynamic memory allocation, and is managed via calls to new, delete, malloc, free, etc.
 - The **stack** is used for local variables. Space on the stack is reserved for local variables when they are declared (at function entrance or elsewhere, depending on the language), and the space is freed up when the variables go out of scope.
 - When processes are swapped out of memory and later restored, additional information must also be stored and restored. Key among them are the program counter and the value of all program registers.



Process State

- Processes may be in one of 5 states, as shown below.
 - New** - The process is in the stage of being created.
 - Ready** - The process has all the resources available that it needs to run, but the CPU is not currently working on this process's instructions.
 - Running** - The CPU is working on this process's instructions.
 - Waiting** - The process cannot run at the moment, because it is waiting for some resource to become available or for some event to occur. For example, the process may be waiting for keyboard input, disk access request, inter-process messages, a timer to go off, or a child process to finish.
 - Terminated** - The process has completed.

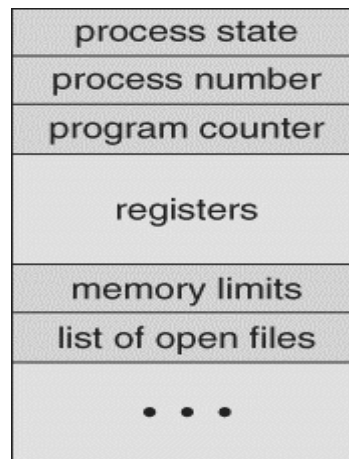


Process Control Block

For each process there is a Process Control Block, PCB, which stores the following (types of) process-specific information, as illustrated in Figure

- Process State** - Running, waiting, etc., as discussed above.
- Process ID**, and parent process ID.
- CPU registers and Program Counter** - These need to be saved and restored when swapping processes in and out of the CPU.
- CPU-Scheduling information** - Such as priority information and pointers to scheduling queues.
- Memory-Management information** - E.g. page tables or segment tables.

- **Accounting information** - user and kernel CPU time consumed, account numbers, limits, etc.
- **I/O Status information** - Devices allocated, open file tables, etc.



Operations on the Process

1. Creation

Once the process is created, it will be ready and come into the ready queue (main memory) and will be ready for the execution.

2. Scheduling

Out of the many processes present in the ready queue, the Operating system chooses one process and start executing it. Selecting the process which is to be executed next, is known as scheduling.

3. Execution

Once the process is scheduled for the execution, the processor starts executing it. Process may come to the blocked or wait state during the execution then in that case the processor starts executing the other processes.

4. Deletion/killing

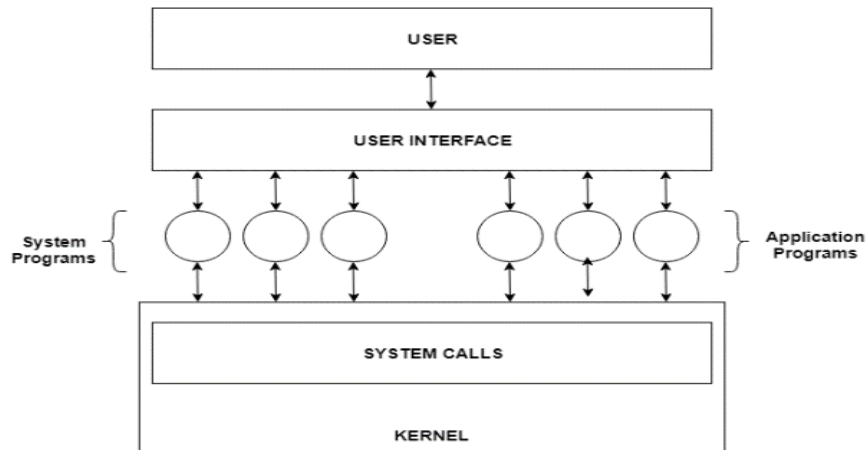
Once the purpose of the process gets over then the OS will kill the process. The Context of the process (PCB) will be deleted and the process gets terminated by the Operating system.

2.4 Discuss and detail about system programs?[5] (NOV'14)

System programs provide an environment where programs can be developed and executed. In the simplest sense, system programs also provide a bridge between the user interface and system calls. In reality, they are much more complex. For example, a compiler is a complex system program.

System Programs Purpose

The system program serves as a part of the operating system. It traditionally lies between the user interface and the system calls. The user view of the system is actually defined by system programs and not system calls because that is what they interact with and system programs are closer to the user interface.



In the above image, system programs as well as application programs form a bridge between the user interface and the system calls. So, from the user view the operating system observed is actually the system programs and not the system calls.

Types of System Programs

System programs can be divided into seven parts. These are given as follows:

Status Information

The status information system programs provide required data on the current or past status of the system. This may include the system date, system time, available memory in system, disk space, logged in users etc.

Communications

These system programs are needed for system communications such as web browsers. Web browsers allow systems to communicate and access information from the network as required.

File Manipulation

These system programs are used to manipulate system files. This can be done using various commands like create, delete, copy, rename, print etc. These commands can create files, delete files, copy the contents of one file into another, rename files, print them etc.

Program Loading and Execution

The system programs that deal with program loading and execution make sure that programs can be loaded into memory and executed correctly. Loaders and Linkers are a prime example of this type of system programs.

File Modification

System programs that are used for file modification basically change the data in the file or modify it in some other way. Text editors are a big example of file modification system programs.

Application Programs

Application programs can perform a wide range of services as per the needs of the users. These include programs for database systems, word processors, plotting tools, spreadsheets, games, scientific applications etc.

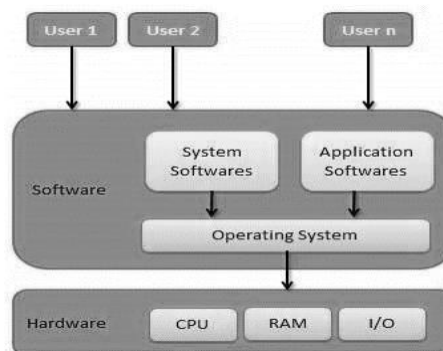
Programming Language Support

These system programs provide additional support features for different programming languages. Some examples of these are compilers, debuggers etc. These compile a program and make sure it is error free respectively.

2.5 List out and discuss operating System Components? (APR'15)**Operating system components**

An operating system is a program that acts as an interface between the user and the computer hardware and controls the execution of all kinds of programs.

An operating system provides the environment within which programs are executed. To construct such an environment, the system is partitioned into small modules with a well-defined interface.



Following are some of important functions of an operating System.

- Memory Management
- Processor Management
- Device Management
- File Management
- Security
- Control over system performance
- Job accounting
- Error detecting aids
- Coordination between other software and users

Process Management The CPU executes a large number of programs. While its main concern is the execution of user programs, the CPU is also needed for other system activities. These activities are called processes. A process is a program in execution

In general, a process will need certain resources such as CPU time, memory, files, I/O devices, etc., to accomplish its task. These resources are given to the process when it is created. In addition to the various physical and logical resources that a process obtains when it is created, some initialization data (input) may be passed along.

A process is the unit of work in a system. Such a system consists of a collection of processes, some of which are operating system processes, those that execute system code, and the rest being user processes, those that execute user code.

The operating system is responsible for the following activities in connection with processes managed.

- The creation and deletion of both user and system processes
- The suspension and resumption of processes.
- The provision of mechanisms for process synchronization
- The provision of mechanisms for deadlock handling.

Memory Management

Memory is central to the operation of a modern computer system. Memory is a large array of words or bytes, each with its own address. Interaction is achieved through a sequence of reads or writes of specific memory address. The CPU fetches from and stores in memory.

In order for a program to be executed it must be mapped to absolute addresses and loaded in to memory. As the program executes, it accesses program instructions and data from memory by generating these absolute is declared available, and the next program may be loaded and executed.

In order to improve both the utilization of CPU and the speed of the computer's response to its users, several processes must be kept in memory.

The operating system is responsible for the following activities in connection with memory management.

- Keep track of which parts of memory are currently being used and by whom.
- Decide which processes are to be loaded into memory when memory space becomes available.
- Allocate and deallocate memory space as needed.

Secondary Storage Management [Disk Management]

The main purpose of a computer system is to execute programs. These programs, together with the data they access, must be in main memory during execution. Since the main memory is too small to permanently accommodate all data and program, the computer system must provide secondary storage to backup main memory.

Most modern computer systems use disks as the primary on-line storage of information, of both programs and data. Hence the proper management of disk storage is of central importance to a computer system.

The operating system is responsible for the following activities in connection with disk management

- Free space management
- Storage allocation
- Disk scheduling.

I/O System

One of the purposes of an operating system is to hide the peculiarities of specific hardware devices from the user. For example, in Unix, the peculiarities of I/O devices are hidden from the bulk of the operating system itself by the I/O system. The I/O system consists of:

- A buffer caching system
- A general device driver code
- Drivers for specific hardware devices.

Only the device driver knows the peculiarities of a specific device.

File Management

File management is one of the most visible services of an operating system. Computers can store information in several different physical forms; magnetic tape, disk, and drum are the most common forms. Each of these devices has its own characteristics and physical organization.

For convenient use of the computer system, the operating system provides a uniform logical view of information storage. The operating system abstracts from the physical properties of its storage devices to define a logical storage unit, the file. Files are mapped, by the operating system, onto physical devices.

A file is a collection of related information defined by its creator. Commonly, files represent programs (both source and object forms) and data

The operating system implements the abstract concept of the file by managing mass storage device, such as tapes and disks.. Finally, when multiple users have access to files, it may be desirable to control by whom and in what ways files may be accessed.

The operating system is responsible for the following activities in connection with file management:

- The creation and deletion of files
- The creation and deletion of directory
- The support of primitives for manipulating files and directories
- The mapping of files onto disk storage.
- Backup of files on stable (non volatile) storage.

Protection System

The various processes in an operating system must be protected from each other's activities. For that purpose, various mechanisms which can be used to ensure that the files, memory segment, CPU and other resources can be operated on only by those processes that have gained proper authorization from the operating system.

For example, memory addressing hardware ensure that a process can only execute within its own address space. The timer ensure that no process can gain control of the CPU without relinquishing it. Finally, no process is allowed to do its own I/O, to protect the integrity of the various peripheral devices.

Protection refers to a mechanism for controlling the access of programs, processes, or users to the resources defined by a computer controls to be imposed, together with some means of enforcement.

Protection can improve reliability by detecting latent errors at the interfaces between component subsystems. Early detection of interface errors can often prevent contamination of a healthy subsystem by a subsystem that is malfunctioning. An unprotected resource cannot defend against use (or misuse) by an unauthorized or incompetent user.

Networking

A distributed system is a collection of processors that do not share memory or a clock. Instead, each processor has its own local memory, and the processors communicate with each other through various communication lines, such as high speed buses or telephone lines. Distributed systems vary in size and function. They may involve microprocessors, workstations, minicomputers, and large general purpose computer systems.

The processors in the system are connected through a communication network, which can be configured in the number of different ways. The network may be fully or partially connected. The communication network design must consider routing and connection strategies, and the problems of connection and security.

A distributed system provides the user with access to the various resources the system maintains. Access to a shared resource allows computation speed-up, data availability, and reliability.

Command Interpreter System

One of the most important components of an operating system is its command interpreter. The command interpreter is the primary interface between the user and the rest of the system.

Many commands are given to the operating system by control statements. When a new job is started in a batch system or when a user logs-in to a time-shared system, a program which reads and interprets control statements is automatically executed. This program is variously called (1) the control card interpreter, (2) the command line interpreter, (3) the shell (in Unix), and so on. Its function is quite simple: get the next command statement, and execute it.

The command statement themselves deal with process management, I/O handling, secondary storage management, main memory management, file system access, protection, and networking.

2.6 Write short notes on Hardware Protection (APR'16)

A computer contains various hardware like processor, RAM, monitor etc. So, OS must ensure that these devices remain intact (not directly accessible by the user). It is divided into three categories:

1) CPU Protection

It means that a process should not hog (hold) CPU forever otherwise other processes will not get the process. For that purpose, a timer is introduced to prevent such a situation. A process is given a certain time for execution after which a signal is sent to the process which makes the process to leave CPU. Hence process will not hog the CPU.

2) Memory Protection

There may be multiple processes in the memory so it is possible that one process may try to access other process memory.

To prevent such situation, we use two registers:

1. Base Register
2. Limit Register

Base register store the starting address of the program and **Limit Register** store the size of the process. So, whenever a process wants to access address in memory then it is checked that it can access the memory or not.

3) I/O protection

To ensure CPU protection OS ensure that below case should not occur

- View I/O of other process
- Terminate I/O of another process
- Give priority to a particular process I/O

If an application process wants to access any I/O device then it will be done through system call so that OS will monitor the task. Like In C language `write()` and `read()` is a system call to read and write on file. There are two modes in instruction execute:

1. User mode

The system performs a task on behalf of user application this instruction. In this mode, the user cannot directly access hardware and reference memory.

2. Kernel mode

Whenever a direct access to hardware is required a system call is used by the application program.

2.7 List down various types of storage devices. Illustrate the hierarchy of storage devices based on the speed and size characteristics. [NOV'16]

A memory element is the set of storage devices which stores the binary data in the type of bits. In general, the storage of memory can be classified into two categories such as volatile as well as non-volatile.

Types of Computer Storage

The computer storage unit is divided into three parts. Given below are details about the three types of computer storage:

- **Primary Storage:** This is the direct memory which is accessible to the Central Processing Unit (CPU).
 - This is also known as the main memory and is volatile.
 - This is temporary. As soon as the device turns off or is rebooted, the memory is erased
 - It is smaller in size
 - Primary storage comprises only of Internal memory
 - Examples of primary storage include RAM, cache memory, etc.
 -
- **Secondary Storage:** This type of storage does not have direct accessibility to the Central Processing Unit.
 - The input and output channels are used to connect such storage devices to the computer, as they are mainly external
 - It is non-volatile and larger storage capacity in comparison to primary storage
 - This type of storage is permanent until removed by an external factor
 - It comprises of both internal and external memory
 - Examples of secondary storage are USB drives, floppy disks, etc.
 -
- **Tertiary Memory:** This type of storage is generally not considered to be important and is generally not a part of personal computers.
 - It involves mounting and unmounting of mass storage data which is removable from a computer device
 - This type of storage holds robotic functions
 - It does not always require human intervention and can function automatically

Characteristics of Storage

Volatility

- Volatile memory needs power to work and loses its data when power is switched off. However, it is quite fast so it is used as primary memory.
- Non - volatile memory retains its data even when power is lost. So, it is used for secondary memory.

Mutability

Mutable storage is both read and write storage and data can be overwritten as required. Primary storage typically contains mutable storage and it is also available in secondary storage nowadays.

Accessibility

Storage access can be random or sequential. In random access, all the data in the storage can be accessed randomly and roughly in the same amount of time. In sequential storage, the data needs to be accessed in sequential order i.e. one after the other.

Addressability

Each storage location in memory has a particular memory address. The data in a particular location can be accessed using its address.

Capacity

The capacity of any storage device is the amount of data it can hold. This is usually represented in the form of bits or bytes.

Performance

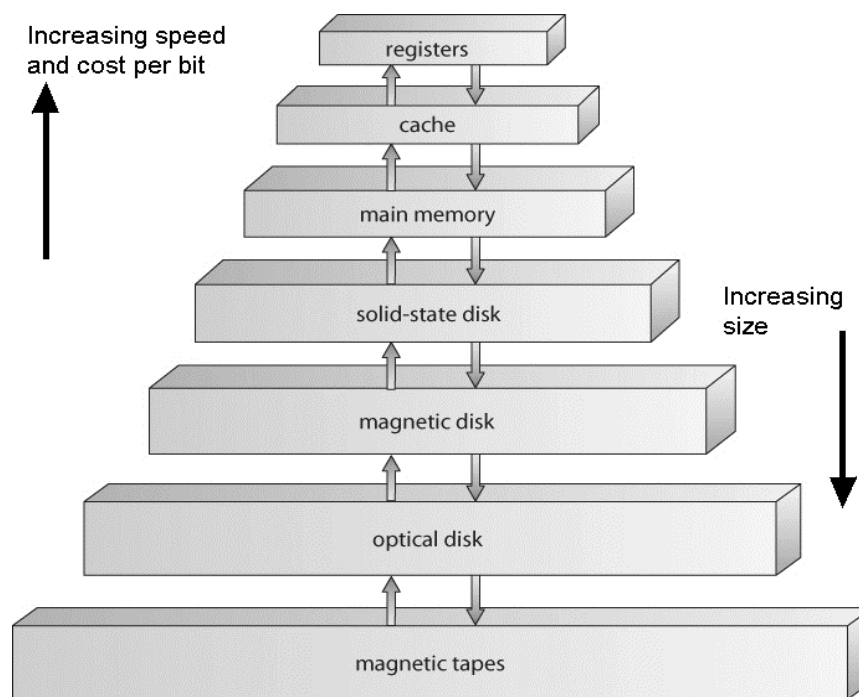
Performance can be described in terms of latency or throughput.

- **Latency** is the time required to access the storage. It is specified in the form of read latency and write latency.
- **Throughput** is the data reading rate for the memory. It can be represented in the form of megabytes per second.

Memory Hierarchy in Computer Architecture

The **memory hierarchy design** in a computer system mainly includes different storage devices. Most of the computers were inbuilt with extra storage to run more powerfully beyond the main memory capacity.

The following **memory hierarchy diagram** is a hierarchical pyramid for computer memory. The designing of the memory hierarchy is divided into two types such as primary (Internal) memory and secondary (External) memory.



Primary Memory

The primary memory is also known as internal memory, and this is accessible by the processor straightly. This memory includes main, cache, as well as CPU registers.

Secondary Memory

The secondary memory is also known as external memory, and this is accessible by the processor through an input/output module. This memory includes an optical disk, magnetic disk, and magnetic tape.

Characteristics of Memory Hierarchy

The memory hierarchy characteristics mainly include the following.

Performance

Previously, the designing of a computer system was done without memory hierarchy, and the speed gap among the main memory as well as the CPU registers enhances because of the huge disparity in access time, which will cause the lower performance of the system. So, the enhancement was mandatory. The enhancement of this was designed in the memory hierarchy model due to the system's performance increase.

Ability

The ability of the memory hierarchy is the total amount of data the memory can store. Because whenever we shift from top to bottom inside the memory hierarchy, then the capacity will increase.

Access Time

The access time in the memory hierarchy is the interval of the time among the data availability as well as request to read or write. Because whenever we shift from top to bottom inside the memory hierarchy, then the access time will increase

Cost per bit

When we shift from bottom to top inside the memory hierarchy, then the cost for each bit will increase which means an internal Memory is expensive compared with external memory.

2.8 What is the purpose of interrupts? What are the differences between a trap and an interrupt? Can traps be generated intentionally by a user program? If so, what is the purpose? (5) (May 2017)

Interrupt is the mechanism by which modules like I/O or memory may interrupt the normal processing by CPU. It may be either clicking a mouse, dragging a cursor, printing a document etc the case where interrupt is getting generated.

Purpose:

External devices are comparatively slower than CPU. So if there is no interrupt CPU would waste a lot of time waiting for external devices to match its speed with that of CPU. This decreases the efficiency of CPU. Hence, interrupt is required to eliminate these limitations.

With Interrupt:

1. Suppose CPU instructs printer to print a certain document.
2. While printer does its task, CPU engaged in executing other tasks.
3. When printer is done with its given work, it tells CPU that it has done with its work.
(The word 'tells' here is interrupt which sends one message that printer has done its work successfully.).

The purpose of interrupts is to alter the flow of execution in response to some event. Without interrupts, a user may have to wait for a given application to have a higher

priority over the CPU to be ran. This ensures that the CPU will deal with the process immediately. In processors with a privileged mode of execution, the interrupt also causes a mode switch into kernel mode.

Advantages:

- It increases the efficiency of CPU.
- It decreases the waiting time of CPU.
- Stops the wastage of instruction cycle.

Disadvantages:

- CPU has to do a lot of work to handle interrupts, resume its previous execution of programs (in short, overhead required to handle the interrupt request.).

Interrupt	Trap
Interrupt is an electronic alerting signal sent to the processor from an external device, either a part of the computer itself such as a disk controller or an external peripheral.	Software Interrupt caused either by an exceptional condition (e.g. divide by zero or invalid memory access) in the processor itself, or a special instruction in the instruction set which causes an interrupt when it is executed is called Trap.
An interrupt transfers control to an interrupt handler inside the operating system; it changes the operating system mode to kernel mode. The interrupt handler is determined using an interrupt vector , which is an array in the operating system that maps interrupt numbers to interrupt handler addresses. An interrupt handler should return the machine to its state before the interrupt, and in particular should change The mode back to user mode.	A trap transfers control to a trap handler in The operating system and changes the mode to kernel mode. The trap handler is determined using a trap vector , which is an array in the operating system that maps trap Numbers to trap handler addresses. Traps are like exceptions in high-level languages
An interrupt can be used to signal the completion of an I/O to obviate the need for device polling.	A trap can be used to call operating system routines Or to catch arithmetic errors.
These are Asynchronous interrupt and may arrive at the execution of any instruction.	These are synchronous interrupt and arrives after the execution of any instruction.

A trap is a software-generated interrupt. A trap can be used to call operating system routines or to catch arithmetic errors. User program can develop a trap which could be formed intentionally. Generally, it is used to find arithmetic errors or rather it is used to call OS routines. Traps can occur through exceptions or through explicit instructions in the program. The reason for this is to allow a user to force a mode switch to kernel mode.